# DoggyLog App Summary

Repo-based, one-page overview

## What It Is

DoggyLog is a Flutter mobile app that combines a pet-focused calendar, countdown tracker, and companion-style pet profile experience in one interface. The repo also includes native hooks for notifications, biometrics, geofencing, iOS calendar sync, and widget snapshot publishing.

## Who It Is For

Primary persona: pet owners, especially dog owners, who want one place to manage care schedules, milestones, reminders, and a lightweight companion avatar.

## What It Does

- Onboards the user by choosing a pet breed and nickname.
- Shows a calendar with week or month views, date selection, and agenda items.
- Creates, edits, completes, and deletes calendar tasks with reminder offsets.
- Tracks countdown items with pinned status, progress, and celebration state.
- Manages pet profiles, active companion selection, loyalty points, and unlockable skins.
- Schedules local notifications for upcoming tasks and supports biometric app unlock.
- Offers iOS calendar sync plus geofence-based scene updates when permissions are granted.

## How To Run

- Install Flutter. Repo evidence: pubspec.yaml targets Dart 3.10.8; local tool check found Flutter 3.38.9.
- From the project root, run: flutter pub get
- Start the app on a simulator or device: flutter run
- Optional verification: flutter test
- Not found in repo: project-specific setup notes beyond the default Flutter starter README.

## How It Works

- UI layer: Flutter screens such as Onboarding, HomeShell, Calendar, Countdown, Settings, and Pets consume appStateProvider.
- State layer: AppStateController bootstraps app data, listens to repository streams, and coordinates reminders, permissions, sync, sensors, and snapshots.
- Data layer: AppRepository uses Drift tables for calendar entries, pets, countdowns, geofences, loyalty ledger, and key-value preferences, plus SharedPreferences for seed flags.
- Platform layer: service wrappers call DoggylogPlatform, which uses a MethodChannel for calendar, notifications, biometrics, sensors, widget snapshots, and Dynamic Island updates.
- Data flow: user action -> controller -> repository/database; state changes -> notification sync and shared snapshot publishing for native surfaces.
- Not found in repo: fully wired CloudKit backup flow. iOS widget extension files exist, but ios/Extensions/README says they are not yet wired into Runner.xcodeproj.

Evidence source set: pubspec.yaml, lib/main.dart, lib/core/bootstrap.dart, lib/app/app.dart, lib/app/router/app_router.dart, lib/features/*, lib/platform/*, ios/Extensions/README.md.